

LeetCode Bitwise Problems

1	LC-136 – Single Number
2	LC-260 – Single Number -III
3	LC-201 – Bitwise AND of number Range
4	LC-693 – Binary Number with Alternating Bits
5	LC-476 – Number Complement
6	LC-191 – Count 1 bits
7	LC-231 – Power of 2
8	LC-190 – Reverse Bits
9	LC-371 – Sum of 2 Integers
10	LC-137 – Single Number -II



LeetCode136.java

//Find One Odd Occurring Number in the Array
//136. Single Number

```
public class LeetCode136 {  
    public static int singleNumber(int[] arr) {  
  
        int result = 0;  
  
        for (int i = 0; i < arr.length; i++) {  
            result = result ^ arr[i];  
        }  
        return result;  
    }  
  
    public static void main(String[] args) {  
        int arr[] = { 3, 5, 5, 6, 6, 7, 8, 8, 3 };  
        int result = singleNumber(arr);  
        System.out.println(result);  
    }  
}
```



LeetCode260.java

//Find Two Odd Occurring Numbers in the Array

//260. Single Number III

```
public class LeetCode260 {
    public static int[] singleNumber(int[] arr) {
        int x = 0;
        for (int i = 0; i < arr.length; i++) {
            x = x ^ arr[i];
        }

        int k = x & (~(x - 1));

        int result[] = new int[2];
        for (int i = 0; i < arr.length; i++) {

            if ((arr[i] & k) != 0)
                result[0] = result[0] ^ arr[i];
            else
                result[1] = result[1] ^ arr[i];
        }
        return result;
    }
    public static void main(String[] args) {
        int arr[] = { 3, 5, 9, 6, 6, 7, 2, 7, 2, 3 };
        int result[] = singleNumber(arr);
        for (int x : result) {
            System.out.println(x);
        }
    }
}
```



LeetCode201.java

// 201. Bitwise AND of Numbers Range

```
public class LeetCode201 {
    public static int rangeBitwiseAnd(int m, int n) {
        int count = 0;
        while(m!=n){
            m >>= 1;
            n >>= 1;
            count ++;
        }
        return m << count;
    }
}
```

LeetCode693.java

//693. Binary Number with Alternating Bits

```
public class LeetCode693 {
    public static boolean hasAlternatingBits(int num) {
        int prevBit = num&1;
        num = num>>1;

        while(num>0) {
            int currBit = num&1;
            if(currBit==prevBit){ return false; }

            prevBit = currBit;
            num = num>>1;
        }
        return true;
    }
}
```



LeetCode476.java

```
package com.myjlc.bitwise.problems;
```

//476. Number Complement

```
public class LeetCode476 {  
  
    public static int findComplement(int num) {  
        int result=0;  
        int i=0;  
        while(num!=0) {  
            if((num & 1) ==0) {  
                result = result + (1<<i);  
            }  
            i++;  
            num =num >>1;  
        }  
        return result;  
    }  
  
    public static void main(String[] args) {  
  
        int result = findComplement(5);  
        System.out.println(result);  
    }  
}
```



LeetCode191.java

// Count Set Bits in the given number

```
public class LeetCode191 {  
    public static int hammingWeight(int n) {  
        int count = 0;  
        while (n!=0) {  
            n = n & (n - 1);  
            count++;  
        }  
        return count;  
    }  
    public static void main(String[] args) {  
        int count = hammingWeight(15);  
        System.out.println(count);  
    }  
}
```

LeetCode231.java

//231. Power of Two

```
class LeetCode231 {  
    public static boolean isPowerOfTwo(int n) {  
        return n>0 && (n & (n - 1)) == 0;  
    }  
    public static void main(String[] args) {  
        boolean flag = isPowerOfTwo(-2147483648);  
        System.out.println(flag);  
    }  
}
```



LeetCode190.java

//Reverse the bits of Given Number

```
public class LeetCode190 {
    public static int reverseBits(int num) {
        int result = 0;
        for (int i = 1; i <= 32; i++) {
            result = result << 1;

            if ((num & 1) > 0)
                result = result + 1;
            num = num >> 1;
        }
        return result;
    }
}
```

LeetCode371.java

//Add Two Numbers without + Operator

```
public class LeetCode371 {
    public static int getSum(int a, int b) {
        while (b != 0) {
            int temp = (a & b) << 1;
            a = a ^ b;
            b = temp;
        }

        return a;
    }
}
```



LeetCode137.java

//137. Single Number II

```
public class LeetCode137 {  
  
    public static int singleNumber(int[] nums) {  
  
        int ones = 0;  
        int twos = 0;  
        for(int num:nums) {  
            ones = ((ones ^ num) & (~twos));  
            twos = ((twos ^ num) & (~ones));  
        }  
        return ones;  
    }  
  
    public static void main(String[] args) {  
        int arr[] = { 5, 5, 5, 9, 6, 6, 6 };  
        int result = singleNumber(arr);  
        System.out.println(result);  
    }  
}
```